RFP Sentinel

Project Overview, Architecture, Domain Findings & Design Rationale — v1

A RAG-based bid-evaluation co-pilot for GeM (Government e-Marketplace) Technical Evaluators, scoped to the **Electronics category**. It serves the **buyer/evaluator side** of government procurement — the gap in a market where every commercial RFP-AI tool serves bidders instead.

A buyer uploads an RFP/tender PDF. The system extracts its requirements, checks them against a knowledge base of real government procurement norms, flags anything that conflicts with a citation, and pauses for a human to review before anything is published. Bidders will later upload their own documents to be checked against that same RFP, in isolation from every other bidder.

Contents

1. Current Status & Architecture

Built and working end-to-end LIVE

- Norm knowledge base — 6 real government documents, 968 chunks (see Section 6)
- RFP PDF upload → criteria extraction → compliance check → human checkpoint (with override + reasoning) → publish
- Bid ingestion — a bidder's multi-document submission (8-10 PDFs) → Qdrant `bids` collection, isolated by `bid_id`, soft-delete lifecycle
- React dashboard (buyer role), FastAPI backend, LangGraph orchestrating the RFP pipeline, Postgres as checkpoint store, minimal JWT login

Not yet built PLANNED

- Evidence-extraction agent (bid content vs. RFP criteria) and the deterministic scoring engine
- **Packet-I/Packet-II separation** in bid ingestion — a real regulatory requirement surfaced by domain research (Section 6), not yet reflected in `ingest_bid.py`'s schema
- Bidder-facing UI, Checkpoint B, admin dashboard, multi-user auth
- The purge script for permanently deleting closed RFPs/bids after their retention window

Architecture

```
Norm PDFs ──▶ Ingestion pipeline (extract→chunk→embed) ──▶ Qdrant `norms` (permanent rulebook)

RFP PDF ──▶ LangGraph: extract_rfp_criteria (rule-based)
             → check_rfp_compliance (real agent: search+threshold-check/LLM)
             → checkpoint_a (pause for human, override+reasoning supported)
             → Postgres stores approved criteria permanently (never touches Qdrant)

Bid PDFs (8-10/bidder) ──▶ ingest_bid.py ──▶ Qdrant `bids`, tagged bid_id/rfp_id/source_file
  [PLANNED] per criterion (from Postgres) → search this bid (Qdrant, bid_id filter) → verdict
```

Design notes

- **Extraction is rule-based** (regex/keywords), not LLM — deliberate speed/reliability trade-off.
- **Numeric checks are deterministic**, not LLM-judged — the LLM reliably draws the wrong conclusion from correct numbers, even at temperature 0.
- **Every compliance flag carries a citation** — never surfaced ungrounded. The actual validation method, absent a labeled dataset.
- **A human override is first-class and audited** — publishing despite a flag requires a stored reasoning, never a silent dismissal.

2. Complete File Map

Three pipelines share the same extraction/chunking/embedding code — only what happens after, and where the result lands, differs.

Pipeline A — Norm knowledge base

Step	File	Function
Extract text/tables	`ingestion/extract_text.py`, `extract_tables.py`	`extract_text_by_page()` (now takes `use_text_flow`, `page_range`), `extract_tables_by_page()`
Strip Hindi	`ingestion/language_filter.py`	`filter_english()`
Chunk	`ingestion/chunker.py`	`chunk_document()`
Embed + store	`backend/rag/embeddings.py`, `qdrant_client.py`	`embed_texts()`, `upsert_chunks()`, `search_active()`, `mark_status()`
Orchestrator	`ingestion/ingest_norms.py`	reads `manifest.json`, runs the chain per norm

Pipeline B — RFP evaluation

Step	File	Function
Extract + classify (rule-based)	`backend/graph/extract_rfp_criteria.py`	`extract_rfp_criteria()`, `_infer_mandatory()`, `_infer_category()`
Deterministic number check	`backend/scoring/threshold_check.py`	`check_percentage_threshold()`
LLM judgment	`backend/llm/ollama_client.py`	`classify()`
The one real agent	`backend/graph/check_rfp_compliance.py`	`check_rfp_compliance()`
Orchestration	`backend/graph/build_graph.py`, `state.py`	`build_graph()`, `EvaluationState`

Pipeline C — Bid ingestion

Step	File	Function
Extract+chunk+embed one bid's files	`ingestion/ingest_bid.py`	`ingest_bid()`
Bids collection, isolation, soft-delete	`backend/rag/qdrant_client.py`	`ensure_bids_collection()`, `search_bid()`, `close_bid()`

Web wiring

Step	File
Login / JWT / endpoints / CORS	`backend/auth.py`, `api/auth.py`, `api/rfp.py`, `main.py`
Upload, dashboard, results UI	`frontend/src/components/RfpUploadForm.jsx`, `pages/BuyerDashboard.jsx`, `components/EvaluationResult.jsx`

3. Tech Stack — Choices & Reasoning

Layer	Choice	Why, and the honest trade-off
LLM + embeddings	Llama 3.2 3B + nomic-embed-text, Ollama, local	Local-first is a hard constraint (bidder data confidentiality, no GPU). Trade-off: less reliable than a frontier model (proven at numeric reasoning), slow (15-25 min/RFP).
Vector store	Qdrant	Self-hosted, first-class payload filtering (`status` , `bid_id` isolation are core, not bolted on). pgvector would avoid a second DB — chosen anyway for filtering/scale built for this pattern.
Metadata/checkpoint store	Postgres	SQLite locks on write; Postgres chosen from v1 to avoid a painful migration when multi-user auth arrives.
Orchestration	LangGraph	`interrupt()` + a real checkpointer = pause indefinitely for a human, resume exactly where it stopped, without hand-building that persistence.
Backend	FastAPI	Native Pydantic reuse, `BackgroundTasks` fit "kick off a 15-min job, poll for status" exactly.
Frontend	React + Vite + Tailwind v4 + Framer Motion	Streamlit can't do a real login/routing/branded UI. Replaced it, kept it at `frontend/legacy_streamlit/` as honest history.

4. Agent Architecture

#	Name	Status
1	RFP Criteria Extraction	**Not an agent** — pure regex/keyword rules, zero LLM calls.
2	RFP Compliance Agent	**Yes — the one real agent.** Built, tested, running.
3	Bid Evidence Extraction Agent	**Designed, not built.** Reuses Agent 2's exact tool pattern.
—	Deterministic Scoring Engine	**Not an agent** — fixed formula, deliberately.

The bar: **uses more than one tool, and its output changes depending on what it discovers** — not "it's a pipeline step."

> *"Criteria extraction is plain rule-based code — no LLM, chosen for speed and reliability under time pressure. The one real agent is the compliance check: it retrieves the relevant rule via vector search, then picks between a deterministic checker for number comparisons and an LLM for judgments needing real reading comprehension — that tool choice plus a verdict depending on what's retrieved is what makes it an agent. The final ranking step is deliberately not an agent either — an AI-computed score isn't defensible in a government audit."*

5. Bid Evaluation Design

Storage split

RFP criteria: Postgres only, never re-embedded. Bid documents: Qdrant, permanent until closed. Analogy: a library search — you don't pre-index your query, only the books.

Comparison direction — criteria drive the loop

Loop over the RFP's fixed criteria list, search the bid for each — not the reverse. Guarantees completeness (a bidder who omitted something correctly returns `not_found`), bounds cost (one LLM call per criterion, known in advance), and matches the scoring engine's needs exactly.

Missing-documents checklist = a filter, not new state

Of `pass` / `fail` / `partial` / `not_found`, filtering to `not_found` *is* the checklist — same pattern as "criteria that need attention" at Checkpoint A.

Isolation

Every chunk tagged `bid_id`; every search filtered to it. Proven: real `bid_id` → results, fake one → zero. Point IDs are deterministic (`uuid5` of `bid_id:source_file:chunk_index`) — safe under concurrent ingestion of many bids, since each is a pure calculation with no shared counter to race on.

Lifecycle

Soft-delete (`status: closed` + timestamp) when the winner is confirmed → 7-day grace window, fully readable → a standalone purge script (not a live scheduler) deletes for real after that. `close_bid()` is built and verified; the purge script is not.

6. Domain Research Findings

Studied the actual GeM/GFR procurement process against the real NIELIT bid already in this project, verifying every concrete claim against primary sources before acting on it.

The Two-Packet system is real, and not yet built into the code

GFR Rule 189 requires technical (Packet-I) and financial (Packet-II) submissions to stay sealed from each other until technical evaluation concludes — the entire point being that price can never influence a compliance judgment. `ingest_bid.py` currently treats a bidder's 8-10 files as one undifferentiated bundle, with nothing preventing financial content from being visible during technical checking. **Design decided, not yet implemented:** the bidder tags each file Packet-I/Packet-II at upload; every search during technical evaluation is permanently hard-filtered to Packet-I only — the same kind of always-on filter already proving out `bid_id` isolation.

ATC clauses can override structured fields — verified twice against the real bid

A GeM bid document isn't flat prose — sections 1-5 are generated from structured form fields, but section 6 (buyer-added ATC) is free text that can explicitly supersede them. Confirmed directly, word-for-word, in two places in the real NIELIT PDF: an ATC splitting-quantity clause states it *"supersedes"* the general MSE Purchase Preference clause elsewhere in the same document, and a separate ATC clause states it *"supersedes the single place Consignee details mentioned anywhere in the Bid document."* Current extraction treats every clause as an unrelated flat item — a real, demonstrated gap. Simple fix agreed: detect supersession language ("supersede," "overrides") and flag it for human attention at Checkpoint A, rather than attempt automatic resolution.

MSME price-band economics — verified with real numbers from the bid

Three different percentages exist across three documents, not a contradiction once the hierarchy is understood: the base MSME Policy Order caps per-tender price-matching at "up to 20%"; this RFP's own default MSE Purchase Preference clause sets 25% (buyer-configured); but this RFP's own Buyer Added ATC explicitly overrides both with a 60:40 split (40% share for a matching MSE within L1+15%). **Design implication:** the future scoring engine can never hardcode a percentage — it must read the specific RFP's own override clause first, falling back to the general default only if none exists.

Scope boundary — a decision still open

Domain research argues RFP Sentinel should stop entirely at a technically-qualified, ranked-by-compliance shortlist — everything from Packet-II opening onward (price ranking, MSE price-matching, award) sits outside the system, since it's deterministic arithmetic the GeM portal already performs. This is a larger scope cut than originally planned (which had Stage 2 price-ranking inside scope) and has not yet been formally adopted — a decision to make before the scoring engine is built, not after.

Knowledge base expanded: 3 → 6 documents, 724 → 968 chunks

Added GFR 2017 (Chapter 6 only — pages 40-56 of 208, the rest is unrelated budget/audit content) and both DPIIT Make-in-India orders (2020-06-04 revision + 2020-09-15 partial amendment, both active simultaneously). One real bug found and fixed: the June DPIIT order renders every line twice for a fake-bold effect, and pdfplumber's default extraction interleaved both passes character-by-character, doubling every letter — fixed

with a verified, opt-in `use_text_flow` flag per document, not a global change, since it was confirmed to alter extraction order on already-working documents too.

The OCR gap is now a real, recurring blocker — not hypothetical

A second real document (the MSME/Make-in-India "concurrent application" memo, cited twice in the real RFP) turned out to be a pure scanned image with zero extractable text. Combined with the earlier 2018 MSME Amendment scan, this is now a proven, repeat blocker — worth prioritizing OCR sooner than originally planned.

7. Local Setup & Running

One-time setup

```
cp .env.example .env
python3 -m venv venv && ./venv/bin/pip install -r requirements.txt
cd frontend && npm install && cd ..
```

Running (three terminals)

```
\# Terminal 1
docker compose up -d && systemctl is-active ollama

\# Terminal 2
./venv/bin/uvicorn backend.main:app --reload --host 0.0.0.0

\# Terminal 3
cd frontend && npm run dev
```

Open **http://localhost:5173**, log in (`buyer@rfpsentinel.local` / `changeme`), upload an RFP PDF.

Bid ingestion (standalone, no UI yet)

```
python -m ingestion.ingest_bid <rfp_id> "<bidder name>" file1.pdf file2.pdf ...
```

Troubleshooting a stuck evaluation

```
ps aux --sort=-%cpu | head -10   \# ollama near the top = genuinely running
pkill -9 -f "uvicorn backend.main:app"   \# to abort
./venv/bin/uvicorn backend.main:app --reload --host 0.0.0.0
```